

→ Approach ←

Old version

- single notebook
- ~58% mAP50

- contaminated taxonomy

this was actually
the most important
issue

clean
data first

document it well
reproducible
config-driven training
(SAM2)

New version

Phase 0:

- audit the dataset.
 - identify classes clearly and harmonize the taxonomy
- 16 → 10 classes. → removed duplicates.

Phase 1:

- new repo, restructured

↳ src → code

↳ data → datasets.

↳ configs → hyperparameters.

↳ runs → output.

! location
independent
scripts.

- write README

- dependencies on req.txt

Phase 2:

- training config → /base.yaml

- train + eval scripts!

P / R / mAP50 / mAP50-95

? → baseline run ⇒ rerun the new model on clean data

↳ check / compare to 58% mAP50

!!! → model improvements (maybe?)
from n to m/s

→ the model didn't improve well enough (63% on vehicles)

↳ re-split data properly → e.g. person = 2 train / 30 tests

car, tank, truck, saw, r520 → these giving really
good results

↳ 0.63 avg

! vehicle, person, trench → really BAD!

OVERALL → 0.447.

⇒ not a trustworthy result.

→ overall test went 0.447 → 0.649

especially person and plane classes are improved

a lot!

same config just changed splitting train/val/test split

just change which
ing goes to which
split

→ we will take this result as our

"first trustworthy number"

Lesson Learned: gained ~ 0.2% just fixing, cleaning the data
data quality and integrity matter

⇒ NEW APPROACH with yoloV8s

- yolov8n $\rightarrow$ yolov8s

- data split and stuff unchanged

$\rightarrow$ we have 3 new classes that actually lowers the overall result

$\hookrightarrow$ worth to mention later.

as of 7/6/2026 7:00pm.

I've tried yolov8s-1280 upscaling the images
the model didn't improve and gave the same result

upscaling to 1280 helped the small classes such as:

person, trench

but hurt the result of large ones

! Lesson $\rightarrow$ Resolution is a scale tradeoff, not an upgrade

What to do now?

- clean and check the taxonomy again

$\rightarrow$ I'm gonna do all classes vs. core classes.

- we changed model + resolution i.e., just changing model and keep res. same to see the actual comparison

n@640 vs. s@1280

- training recipe examination.

core means
"enough instances"

n@640 was 0.726
s@1280 is 0.671 } big prob.

$\Rightarrow$ s@640 is our new best model

all classes $\rightarrow 0.688$

core-8 classes $\rightarrow 0.757$

$\rightarrow$ model capacity does help $\Rightarrow n \rightarrow 51 + 0.03$

$\rightarrow$ high resolution does hurt the model

$\hookrightarrow s@640 + 0.067 = s@1280$

data split fix $>$ capacity $>$ resolution

one-variable
experiment gave
the correct
conclusion;
not the two-var.
experiment.

next step

recipe examining

- dropout

$\rightarrow$ regularization that prevents overfitting

- mixup

$\rightarrow$ data aug. + regularization that synthesise
a new sample by combining 2 existing
data points

$\rightarrow$ dropping dropout since it wasn't consider
all this time in model evaluation

no-op for detection

only applied in CLASSIFICATION

approach $\rightarrow$ disable dropout

$\rightarrow$ reduce mixup to 0.0

$\Rightarrow$ RESULTS!

$\Rightarrow$ real table all the way from beginning

+0.2 (resplit) n@640 - 0.649 - 0.726

→ biggest wins come from data quality and training recipe

-0.3 (model + res charge) s@1280 - 0.621 - 0.671

+0.104 (model charge) s@640 - 0.638 - 0.757

+0.07 (recipe ex) s@640 - 0.757 - 0.795 → no mixup (=0.0)
no dropout.

⇒ "aggressive augmentation is choking a fine-tune"

→ recall jumped from 0.638 → 0.726

meaning: model finding more objects; not just getting conf.

Best Model so far:

yolo v8s@640 with no mixup / dropout

all classes → 0.757

core classes → 0.795